

TRƯỜNG ĐẠI HỌC CÔNG NGHỆ - ĐHQGHN
KHOA CÔNG NGHỆ THÔNG TIN

BÁO CÁO BÀI TẬP LỚN

HỆ THỐNG ĐẤU GIÁ TRỰC TUYẾN (ONLINE AUCTION SYSTEM)

Môn học: Lập trình nâng cao

Giảng viên : Trương Xuân Hiếu

Giảng viên : Đào Xuân Nghĩa

Nhóm sinh viên thực hiện: 10

1. Đặng Hải Anh (Nhóm trưởng) – MSV : 25020005
2. Vũ Anh Kiệt – MSV : 25020208
3. Đặng Thảo Vy – MSV : 25020436
4. Mai Ngọc Hùng – MSV : 25020172

I. Giới thiệu mục tiêu và phạm vi thực hiện

- 1. Mục tiêu:

Xây dựng hoàn chỉnh một Hệ thống Đấu giá Trực tuyến (Online Auction System) sử dụng ngôn ngữ Java, đáp ứng các tiêu chuẩn khắt khe về thiết kế phần mềm.

Áp dụng các nguyên lý Lập trình hướng đối tượng (OOP) và các Design Patterns (Singleton, Factory, Observer) nhằm tạo ra mã nguồn linh hoạt, dễ bảo trì và dễ mở rộng.

Làm chủ kỹ năng lập trình mạng (Socket), lập trình đa luồng (Concurrency) để xử lý các phiên đấu giá theo thời gian thực (real-time) với độ trễ thấp.

Thiết kế giao diện người dùng trực quan, hiện đại bằng JavaFX kết hợp với mô hình MVC, mang lại trải nghiệm người dùng tối ưu.

- 2. Phạm vi thực hiện:

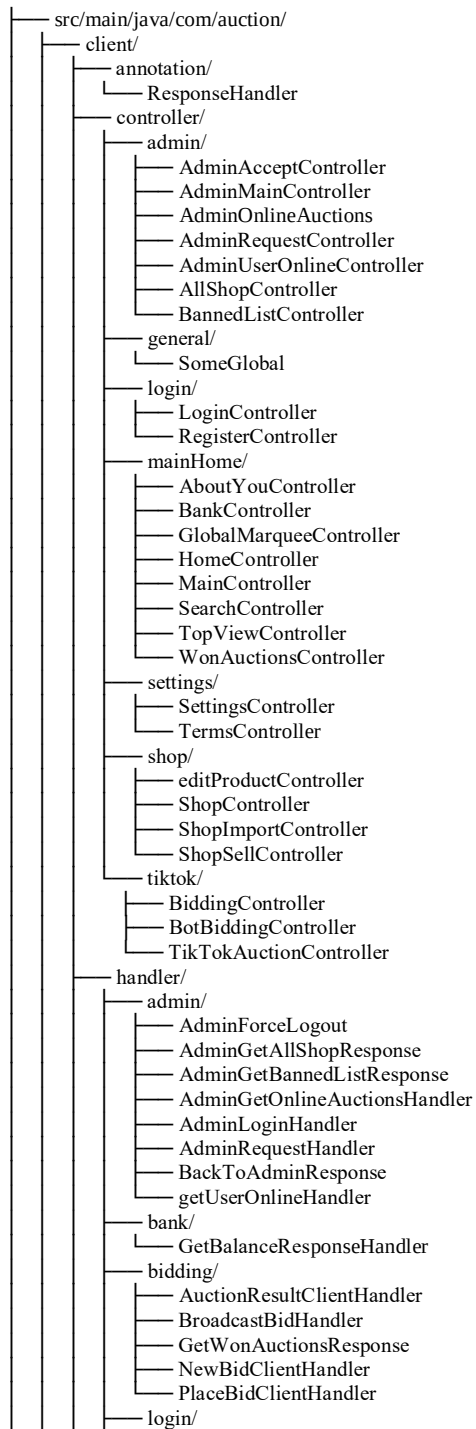
Ứng dụng Desktop toàn diện hỗ trợ 3 vai trò: Admin (Quản trị hệ thống), Seller (Người đăng bán sản phẩm), và Bidder (Người tham gia đấu giá).

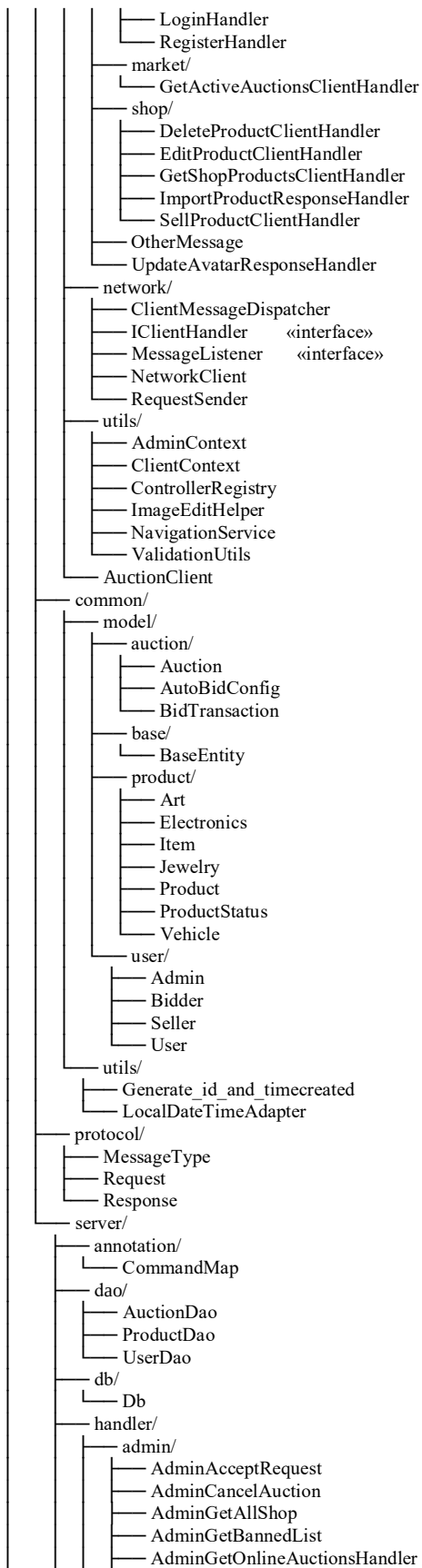
Hệ thống bao gồm các Module cốt lõi: Quản lý người dùng, Quản lý sản phẩm (CRUD), Đặt giá thời gian thực, Tự động phân định kết quả phiên đấu giá, và Hệ thống xử lý ngoại lệ an toàn.

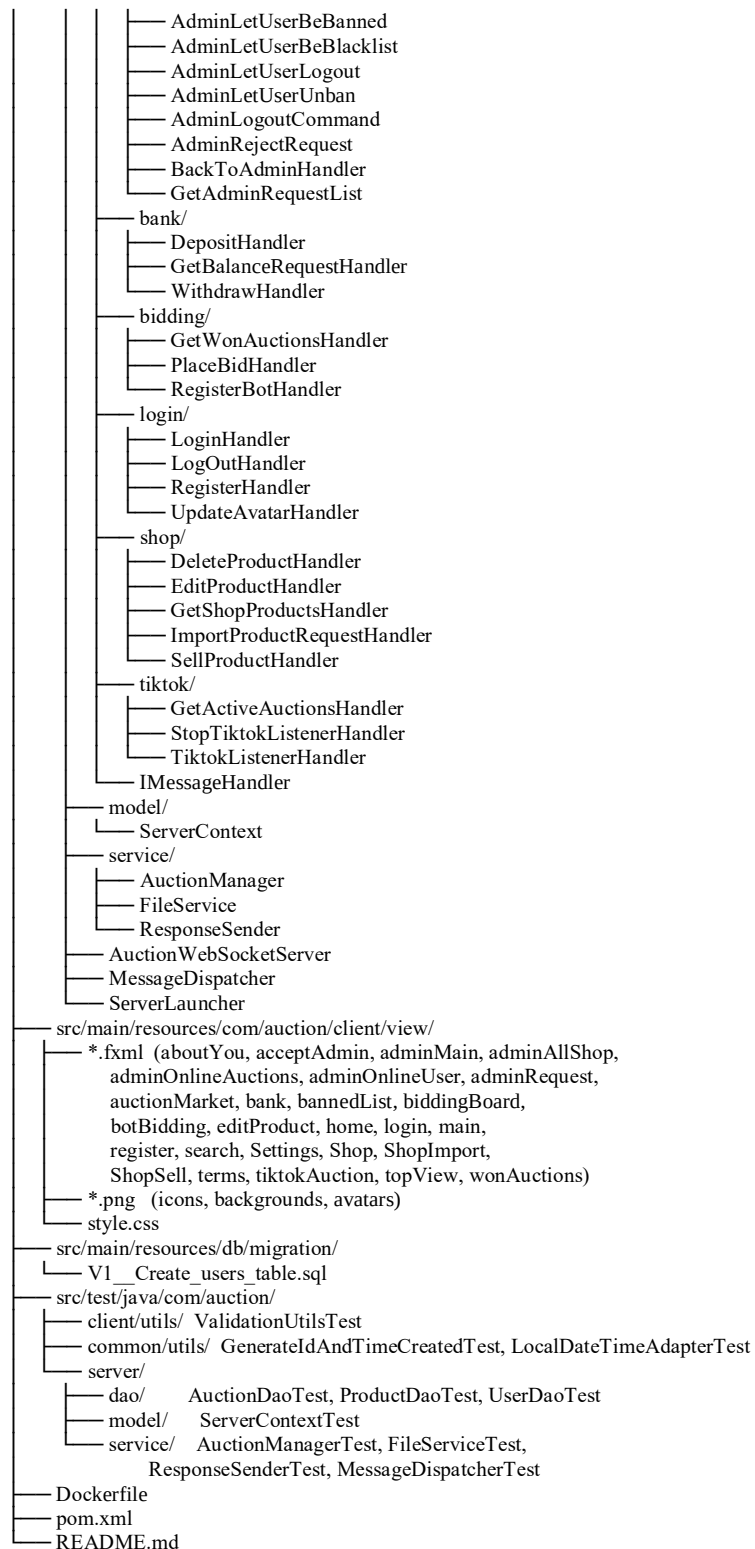
Triển khai trên kiến trúc Client-Server thông qua kết nối Socket cục bộ hoặc qua mạng internet.

II. Kiến trúc tổng thể của hệ thống

BTL/







Hệ thống được thiết kế theo kiến trúc Client-Server kết hợp chặt chẽ với mô hình MVC (Model-View-Controller) tại phía Client, đảm bảo sự tách biệt giữa logic hiển thị và xử lý dữ liệu.

- 1. Tầng Server (Backend & Networking):

Đóng vai trò là trung tâm điều phối, xử lý toàn bộ logic nghiệp vụ (business logic) và lưu trữ dữ liệu thông qua Serialization (File I/O).

Sử dụng ServerSocket để lắng nghe kết nối liên tục. Khi một Client kết nối, Server cấp phát một luồng độc lập (Multi-threading) để xử lý I/O không đồng bộ, tránh tình trạng gián đoạn dịch vụ.

Xử lý đồng thời (Concurrency): Tận dụng từ khoá synchronized và ReentrantLock tại các vùng chia sẻ dữ liệu (ví dụ: biến lưu trữ mức giá bid cao nhất hiện tại). Điều này loại bỏ hoàn toàn các lỗi Race Condition hoặc Lost Update khi hàng chục Bidder gửi lệnh đặt giá cùng một mili-giây.

- 2. Tầng Client (Frontend & MVC):

View: Giao diện được thiết kế độc lập bằng công cụ SceneBuilder, xuất ra các file .fxml. Giao diện trực quan, chia rõ các màn hình quản lý, đăng nhập, và bảng đấu giá live.

Controller: Xử lý sự kiện từ người dùng (nhấn nút, nhập text) và đóng gói dữ liệu gửi qua Socket tới Server.

Model: Các Entity classes ánh xạ dữ liệu nhận về từ Server để render lên UI.

- 3. Cơ chế Real-time (Observer Pattern):

Bất cứ khi nào một phiên đấu giá có biến động (có người trả giá cao hơn, hoặc phiên đấu giá kết thúc), Server đóng vai trò là Subject sẽ lập tức broadcast gói tin trạng thái mới tới tất cả các Client đang theo dõi phiên đấu giá đó. Điều này giúp màn hình đấu giá của mọi Bidder được cập nhật tức thời mà không cần thao tác reload (F5).

III. Các chức năng đạt được và Hướng giải quyết

Dưới đây là đối chiếu các chức năng đã hoàn thành so với yêu cầu đề bài:

Nội dung yêu cầu	Chức năng thực tế	Hướng giải quyết (Giải pháp)	Lý do lựa chọn
Xác định và triển khai các lớp chính (0.5đ)	Quản lý đối tượng cốt lõi của hệ thống.	Xây dựng các Entity class: User, Item, Auction, BidTransaction.	Rõ ràng, dễ map với cơ sở dữ liệu, phân định đúng thực thể.

Áp dụng nguyên tắc OOP (1đ)	Cấu trúc code cho toàn bộ dự án.	Dùng private cho thuộc tính (Đóng gói), abstract class cho hành vi chung (Trừu tượng, Đa hình).	Tối ưu tái sử dụng code, giúp các thành viên dễ dàng maintain.
Quản lý người dùng, sản phẩm (1đ)	Thêm, sửa, xóa, xác thực người dùng/vật phẩm.	Tạo các lớp Controller xử lý logic nghiệp vụ, giao tiếp với DB qua DAO Pattern.	Tách biệt logic xử lý và tầng lưu trữ dữ liệu.
Chức năng đấu giá (1đ)	Đặt giá (Place bid) và cập nhật trạng thái.	Client gửi request chứa thông tin bid qua luồng Socket, Server kiểm tra và cập nhật Auction.	Đáp ứng đúng luồng nghiệp vụ (business logic) cốt lõi của ứng dụng.
Xử lý lỗi & Ngoại lệ (1.0đ)	Chặn giá không hợp lệ, phiên đóng...	Xây dựng các lớp Custom Exceptions kế thừa Exception. Bắt lỗi bằng try-catch và hiển thị Alert Dialog bằng JavaFX.	Giữ ứng dụng không bị Crash khi thao tác sai, tăng độ hoàn thiện UX.
Xử lý đấu giá đồng thời an toàn (1đ)	Xử lý nhiều lượt Bid cùng lúc.	Khoá các method ghi dữ liệu bằng ReentrantLock và synchronized khối xử lý (Critical Section).	Đảm bảo tính toàn vẹn dữ liệu tuyệt đối (Thread-safe), không ai bị mất lượt.
Realtime update (Observer/Socket) (0.5đ)	Cập nhật giá thầu mới nhất liên tục.	Server lưu danh sách Client Sockets. Khi có bid hợp lệ, dùng vòng lặp broadcast thông điệp đến tất cả.	Đảm bảo tính công bằng và minh bạch thông tin cho mọi người chơi.
Thiết kế kiến trúc Client-Server (0.5đ)	Giao tiếp mạng giữa các luồng hệ thống.	Phân tách rõ ràng module Server (xử lý logic) và module Client (xử lý UI), giao tiếp qua TCP Sockets.	Dễ dàng chia nhánh công việc cho 4 người trên GitHub, độc lập phát triển.
Áp dụng MVC (0.5đ)	Quản lý giao diện người dùng.	Sử dụng JavaFX với file .fxml (View), tách biệt hoàn toàn với Controller và Model.	Code giao diện không bị trộn lẫn với logic, dễ debug và sửa đổi UI.
Sử dụng Maven/Gradle, clean code (0.5đ)	Quản lý thư viện và chuẩn mực code.	Khởi tạo project bằng Maven (pom.xml),	Đồng bộ môi trường cho cả đội, dễ dàng tích

		thống nhất quy tắc đặt tên CamelCase.	hợp thư viện bên ngoài.
Unit Test (JUnit) (0.5đ)	Kiểm thử tự động các hàm quan trọng.	Viết test case bằng JUnit 5 cho các phương thức tính toán giá trị bid và kiểm tra ngoại lệ.	Đảm bảo logic cốt lõi chạy đúng trước khi merge code vào nhánh chính.
Thiết lập CI/CD cơ bản (0.5đ)	Tự động hoá luồng kiểm thử khi push code.	Tạo file cấu hình maven.yml trong GitHub Actions để tự động build và chạy test.	Giảm thiểu rủi ro lỗi hệ thống khi gộp code từ nhiều nhánh khác nhau.
Auto-Bidding (0.5đ)	Tự động đặt giá theo cấu hình ngân sách.	Dùng PriorityQueue lưu trữ luồng auto-bid, tự động trigger khi có tài khoản khác đặt giá cao hơn.	Tăng tính tiện ích, giữ chân người dùng không cần theo dõi liên tục.
Bid History Visualization (0.5đ)	Biểu đồ lịch sử biến động giá.	Dùng LineChart của JavaFX, update data series mỗi khi nhận luồng broadcast từ Server.	Giao diện trực quan, chuyên nghiệp, kích thích tâm lý người chơi.
Tiktok auction (0.5đ)	Lướt được sản phẩm như trên tiktok.	Hiển thị pop-up chớp nhoáng cho người thắng.	Bắt kịp xu hướng UX hiện đại, tạo sự hấp dẫn người dùng.

IV. Phân chia công việc

Thành viên	Công việc phụ trách	Mức độ hoàn thành
Đặng Hải Anh	- Thiết kế kiến trúc tổng thể, sơ đồ OOP & Design Patterns Áp dụng mô hình MVC phía Client. - Xử lý Đa luồng (Concurrency) để giải quyết xung đột khi đặt giá đồng thời.	100%
Vũ Anh Kiệt	- Lập trình mạng (Socket Client-Server).	100%

	-Triển cơ chế Realtime Update (Observer Pattern) cho bảng đấu giá. -- Phát triển tính năng mở rộng Auto-Bidding / Anti-sniping (<i>Tính năng tùy chọn</i>).	
Đặng Thảo Vy	- Xử lý lưu trữ dữ liệu (Serialization / File IO). - Xử lý các luồng Ngoại lệ (Exceptions). - Viết Unit Test (JUnit 5), thiết lập pipeline CI/CD trên GitHub. - Tổng hợp slide và viết tài liệu README.	100%
Mai Ngọc Hùng	- Lập trình logic chức năng CRUD cho Người dùng và Sản phẩm. - Tích hợp biểu đồ Bid History Visualization (<i>Tính năng tùy chọn</i>). - Thiết kế giao diện UI/UX bằng JavaFX & SceneBuilder.	100%

Báo cáo được trình bày bám sát yêu cầu, bao phủ toàn bộ barem điểm. Kính mong hội đồng xem xét, đánh giá kết quả thực hiện của nhóm.